MỘC STUDIO / SEMINAR RABBITMQ

Hướng dẫn chạy, lời thuyết trình và giải thích code

Bản ngày 31/10/2026, cấu trúc shop/. Giải thích cho người mới đọc code: chức năng, lý do dùng, hậu quả khi xóa/đổi, câu trả lời GV và vị trí hàm lấy từ source hiện tại.

Chạy nhanh

```
git clone https://github.com/Sunn-dk/demo_soa_rabbitmq.git
cd demo_soa_rabbitmq
docker compose --profile kafka up -d --build --wait --wait-timeout 240
```

Cài Git và Docker Desktop, mở Docker trước. Web: http://localhost:8000. Hộp thư Mailpit: http://localhost:8025. RabbitMQ Management: http://localhost:15672 (guest / guest). Không cần cài Python/Node khi chạy Docker.

Bắt đầu với 0 worker. Đặt đơn, thêm consumer tạo hóa đơn, sau đó thêm consumer gửi email. Hai bước tách biệt để quan sát bất đồng bộ. Mailpit nhận SMTP và file PDF thật trong máy; không gửi thư ra Internet.

Các phần trong bản in

Đọc sơ đồ trước, sau đó sổ tay theo từng bước. Dùng phần câu hỏi để tập trả lời; phụ lục có dòng mã và trích đoạn thật. Kịch bản thao tác vẫn dành cho 15-20 phút; sổ tay dài hơn để học và tra cứu.

Mục lục

MỘC STUDIO / SEMINAR RABBITMQ 1

Hướng dẫn chạy, lời thuyết trình và giải thích code . . . 1

Chạy nhanh . . . 1

Các phần trong bản in . . . 1

Mục lục 2

Sơ đồ file và hai chặng xử lý 4

Sơ đồ email và khôi phục 5

Sổ tay đọc code và trả lời giảng viên 6

1. Đề tài này giải quyết vấn đề thực tế gì? . . . 6

2. Các từ cần hiểu trước khi nhìn code . . . 6

3. Bản đồ file và thứ tự nên đọc . . . 7

4. Từ nút đặt hàng đến HTTP 202 . . . 8

5. Server tính tiền ở đâu và vì sao không tin giá trên trình duyệt? . . . 8

6. SQLite đang giữ gì? Transaction có tác dụng gì? . . . 9

7. Outbox: vì sao không publish ngay sau INSERT là xong? . . . 9

8. RabbitMQ: message được đưa vào đúng queue bằng cách nào? . . . 10

9. Nút thêm worker có mở chương trình thật không? . . . 11

10. Tạo hóa đơn: code nào tạo file và code nào ghi DB? . . . 11

11. Email có thật không, nhận ở đâu và giới hạn chống trùng là gì? . . . 12

12. ACK, prefetch và crash: cần hiểu ba thời điểm . . . 13

13. Lỗi email, DLQ và retry đi qua những hàm nào? . . . 13

14. Kafka: khác ở transport, không đổi nghiệp vụ để thiên vị . . . 14

15. Docker và từng thư viện đảm nhận phần nào? . . . 15

16. Phần nào có thể bỏ, phần nào không nên xóa khi học? . . . 15

17. Hai mươi câu hỏi GV hay xoáy và cách trả lời ngắn . . . 16

18. Cách tập đọc code trong 40 phút . . . 17

19. Nguồn chính thức để kiểm tra lời giải thích . . . 17

Kịch bản Mộc Studio — 15–20 phút 19

0–3 phút: vấn đề thực tế và đặt hàng . . . 19

3–6 phút: tách hai consumer . . . 19

6–9 phút: backlog và thêm worker . . . 19

9–12 phút: tin lỗi và phục hồi . . . 19
12–17 phút: Kafka cùng nghiệp vụ . . . 20
17–20 phút: chốt ý và để lớp thử . . . 20

So sánh cho bài toán tạo hóa đơn và gửi email 21
Điều kiện chung và khác biệt có chủ ý . . . 21
Thí nghiệm 1: partition là đơn vị gán việc của classic group . . . 21
Thí nghiệm 2: lỗi nghiệp vụ khác lỗi broker . . . 21
Thí nghiệm 3: lợi thế log của Kafka . . . 22
Kết luận dùng để thuyết trình . . . 22
Nguồn học chính thức . . . 22

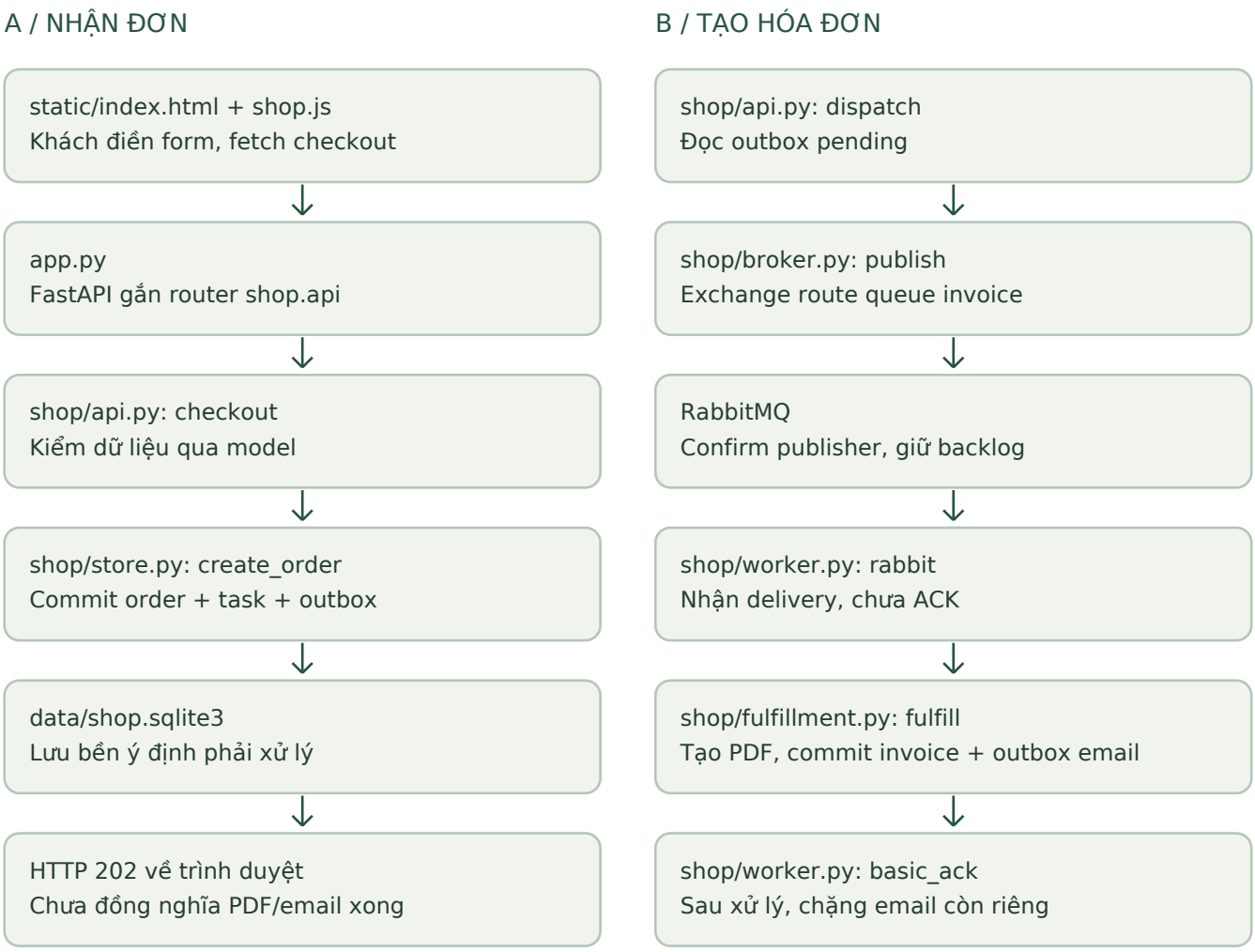
Đánh giá demo sau kiểm thử ngày 31/10/2026 23
Kết luận theo bằng chứng . . . 23
Môi trường và cách thực hiện . . . 23
Những gì đã qua kiểm tra . . . 23
Một sai lệch của kiểm thử đã được sửa . . . 24
Cách trình bày để người nghe thấy ý nghĩa . . . 24
Phạm vi chưa kiểm tra và rủi ro cần nói đúng . . . 25

Phụ lục: mở đúng hàm, đúng dòng 26
app.py | SHA-256 e34b59422960 . . . 26
shop/api.py | SHA-256 6b03d1237cdc . . . 26
shop/store.py | SHA-256 ceeecd67356c . . . 27
shop/broker.py | SHA-256 e75a93208745 . . . 28
shop/worker.py | SHA-256 b34b48c71519 . . . 28
shop/fulfillment.py | SHA-256 9646e6568b30 . . . 28
static/shop.js: điểm vào từ giao diện . . . 28

Phụ lục: trích đoạn mã thật có số dòng 30
HTTP 202 và lưu đơn | shop/api.py . . . 30
Idempotency và transaction order/outbox | shop/store.py . . . 30
Invoice và outbox email cùng transaction | shop/store.py . . . 30
Topology, DLX và persistent publish | shop/broker.py . . . 31
Nhận, xử lý, NACK hoặc ACK | shop/worker.py . . . 31
Dedup invoice và crash sau commit | shop/fulfillment.py . . . 32
Attachment, SMTP và ghi sổ gửi | shop/fulfillment.py . . . 32
Kafka xử lý rồi commit | shop/worker.py . . . 32
Kafka pause partition sau lỗi | shop/worker.py . . . 33

Sơ đồ file và hai chặng xử lý

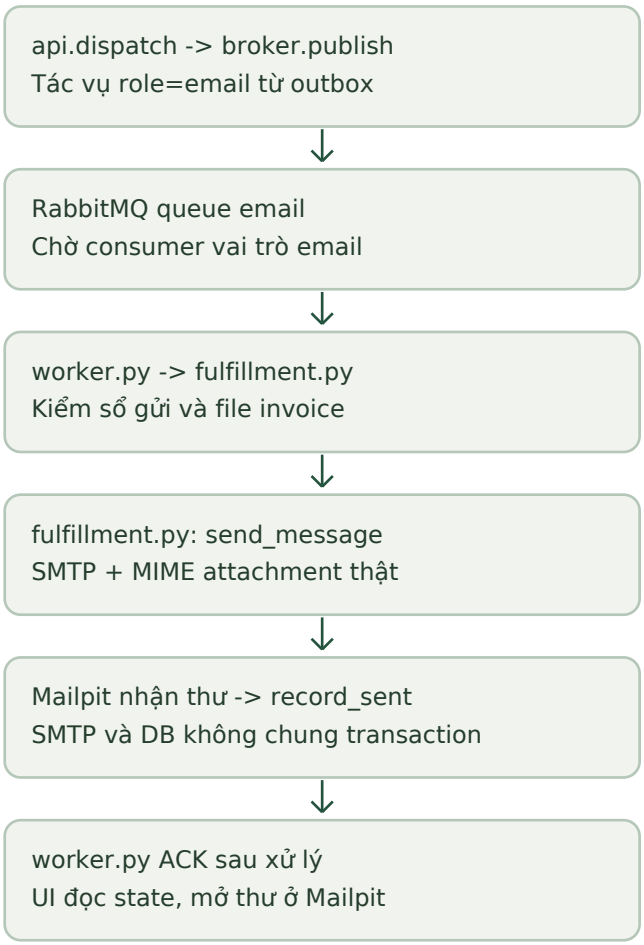
Đường HTTP bên trái chốt đơn trước. Dispatcher bên phải chạy độc lập; nó đọc outbox chứ không đợi người dùng bấm tạo PDF.



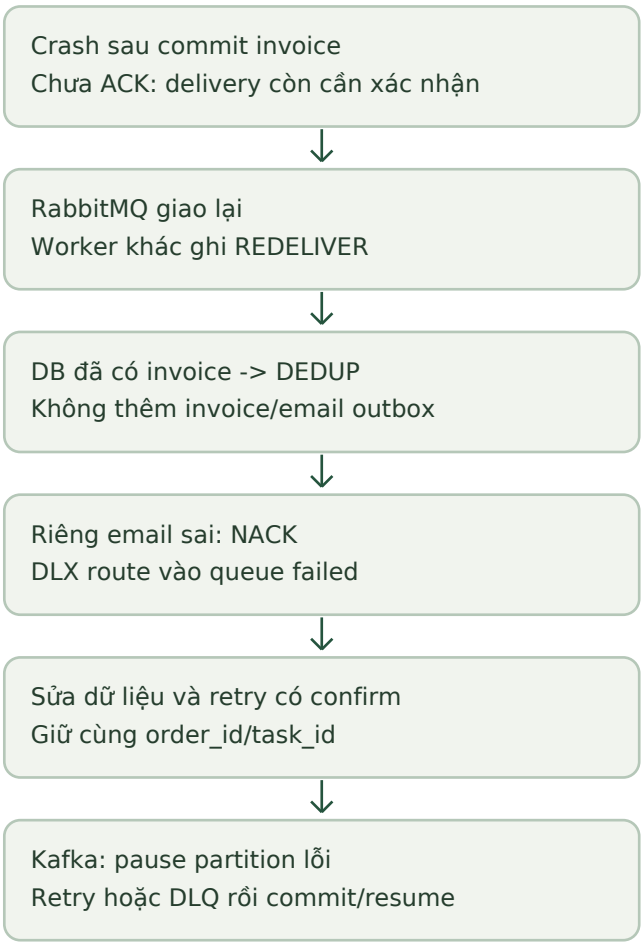
store.py giữ DB; broker.py giữ cách nói chuyện broker; worker.py điều phối nhận/xác nhận; fulfillment.py làm nghiệp vụ. static/shop.css chỉ trình bày. Mũi tên chỉ thứ tự công việc, không phải tất cả đều là lời gọi Python trực tiếp.

Sơ đồ email và khôi phục

C / EMAIL KÈM PDF



D / CRASH VÀ LỖI



Email vẫn có thể lặp nếu SMTP đã nhận nhưng process chết trước khi ghi sổ. Kafka pause ở đây là chính sách giữ thứ tự của app; partition khác không bị bắt buộc dừng. Sơ đồ Mermaid từng nút/file nằm ở docs/LUONG_XU_LY.md.

Sổ tay đọc code và trả lời giảng viên

Dành cho người đã dùng AI hỗ trợ viết code nhưng muốn tự hiểu và giải thích được. Bản ngày 31/10/2026, theo cấu trúc shop/. Không học thuộc tất cả: đọc một đơn hàng đi qua đâu, sau đó hiểu các điểm có thể hỏng. Phụ lục cuối tài liệu chỉ đúng dòng bắt đầu/kết thúc của từng hàm trong mã hiện tại.

1. Đề tài này giải quyết vấn đề thực tế gì?

Khách đặt một đơn gồm sản phẩm, kích cỡ và số lượng. Server phải tính tiền đúng và ghi nhận đơn. Tạo file PDF và gọi máy chủ email là hai việc có thể chậm hoặc lỗi riêng. Nếu API chờ xong cả hai, thời gian khách chờ phụ thuộc toàn bộ quy trình. Ở demo, API lưu đơn và yêu cầu xử lý rồi trả HTTP 202; worker tiếp tục làm phía sau.

Kết quả nghiệp vụ có thật: bản ghi đơn hàng, file PDF có từng dòng hàng và một email SMTP có PDF đính kèm trong Mailpit. Nhịp 2/4/6 giây chỉ làm chậm để nhìn được; đặt 0 thì file và thư vẫn được tạo. Queue không làm phần tạo PDF tự nhanh hơn: nó đệm việc và giúp phân phối cho worker.

Một đơn thưa thớt, PDF rất nhẹ và không cần phục hồi có thể viết đồng bộ đơn giản hơn. Giá trị của queue rõ hơn khi có backlog, cần tăng worker, email hỏng mà vẫn nhận đơn hoặc worker bị dừng giữa chừng. Đó là lý do buổi demo phải có các tình huống, không chỉ bấm nút rồi thấy số tăng.

Câu trả lời ngắn: “Em tách việc nhận đơn khỏi tạo PDF và gửi email. RabbitMQ giữ và phân phối công việc; kết quả cuối là PDF và thư cụ thể. Em chứng minh thêm khả năng xử lý backlog và giao lại khi worker chết.”

2. Các từ cần hiểu trước khi nhìn code

- Process: một chương trình đang chạy với bộ nhớ riêng và PID. Web là một process; mỗi consumer được nút thêm worker mở thành process khác.

- Thread: luồng chạy trong một process. Dispatcher chạy ở thread nền của web; nó khác các process worker.

- HTTP request/response: trình duyệt gửi yêu cầu và server trả câu trả lời của lần gọi đó. HTTP 202 không phải một message ACK của RabbitMQ.

- API/route: cửa vào của server, ví dụ POST /api/shop/checkout; POST thực hiện hành động, GET đọc dữ liệu, PATCH sửa một phần.

- Producer: bên gửi yêu cầu làm việc vào broker. Ở đây dispatcher là producer trực tiếp; handler HTTP tạo yêu cầu trong outbox trước.

- Broker: hệ thống trung gian quản lý truyền/giữ dữ liệu; RabbitMQ và Kafka là hai broker khác mô hình.

- Message/task: phong bì yêu cầu làm việc. order_id chỉ đơn; task_id thêm vai trò, như DH-...:invoice; role chọn invoice/email.

- Exchange/binding/routing key: exchange nhận message và đối chiếu binding; routing key invoice hoặc email chọn queue phù hợp. Exchange không phải nơi lưu backlog lâu dài.

- Queue: nơi chờ của các delivery công việc RabbitMQ. Nhiều consumer cùng queue chia việc; không mặc định mỗi consumer nhận một bản sao.

- Consumer/worker: consumer là vai trò nhận message; worker là process của demo vừa nhận vừa thực hiện nghiệp vụ. Một process không tự đồng nghĩa có một partition Kafka.

- Transaction/commit/rollback: một nhóm thay đổi DB được chốt cùng nhau hoặc hủy cùng nhau. Không suy ra transaction SQLite bao trùm luôn broker, file và SMTP.

- Outbox: bảng lưu yêu cầu cần publish trong cùng transaction với dữ liệu nghiệp vụ. Đây là hộp việc "phải gửi", không phải hộp thư của khách.

- Idempotency: cùng yêu cầu hoặc cùng task được gặp lại nhưng không tạo thêm hiệu ứng nghiệp vụ đã được bảo vệ. Không phải cấm broker giao lặp.

- ACK: consumer xác nhận lượt giao đã xử lý. NACK: từ chối lượt giao; requeue quyết định có đưa lại queue nguồn hay không.

- Publisher confirm: xác nhận của broker với publisher, độc lập với consumer ACK. Có confirm không chứng minh khách đã nhận email.

- Ready/Unacked: Ready còn chờ giao; Unacked đã giao nhưng chưa ACK. Số đếm Management có độ trễ lấy mẫu.

- DLX/DLQ: DLX là exchange route lỗi; DLQ là queue đích giữ tin lỗi. Cần khai báo và binding, không tự có chỉ vì Python ném exception.

- Redelivery: giao lại công việc; dedup: nhận ra hiệu ứng đã có để tránh tạo thêm.

- Topic/partition/offset/group: Kafka lưu log trong topic chia partition; offset là vị trí trong một partition; consumer group giữ tiến độ riêng. Offset không duy nhất trên cả topic.

- SMTP/MIME: SMTP vận chuyển email; MIME biểu diễn nội dung và file đính kèm. Mailpit nhận SMTP trong máy thực hành, không phải Gmail.

3. Bản đồ file và thứ tự nên đọc

Đọc app.py để biết server mở cái gì; static/shop.js để tìm nút; shop/api.py để tìm route tương ứng; shop/store.py để biết dữ liệu được ghi ra sao; shop/broker.py để biết message đi đâu; shop/worker.py để thấy ACK/lỗi; cuối cùng shop/fulfillment.py để xem PDF và email thật.

app.py chỉ gắn router và static files, đồng thời dùng lifespan gọi startup/shutdown. shop/__init__.py giúp Python coi thư mục là package; không có nghiệp vụ. Các file trong tests/ kiểm tra ứng dụng, không nằm trong luồng đặt hàng. tools/build_guide.py chỉ tạo tài liệu in, xóa nó không làm cửa hàng ngừng chạy.

HTML là khung form/nút. JS lấy dữ liệu và điều khiển sự kiện. CSS chỉ trình bày. Không tìm logic ACK trong CSS, cũng không coi trạng thái do JS vẽ là bằng chứng broker đã ACK. Chức năng broker phải đối chiếu callback worker, DB và số đếm broker.

Sơ đồ đầy đủ từng nút và luồng gọi nằm ở docs/LUONG_XU_LY.md. Phần sơ đồ trong PDF phân biệt đường HTTP với đường message; đây là hai loại giao tiếp khác nhau.

4. Từ nút đặt hàng đến HTTP 202

Mở file: static/index.html, form checkout-form; static/shop.js, handler onsubmit; shop/api.py, model Checkout và hàm checkout.

Trình duyệt gom tên/email/địa chỉ và giỏ hàng. JS tạo crypto.randomUUID() cho idempotency key; khi đang gọi thì khóa nút để giảm nhấp đúp. api() dùng fetch, JSON.stringify và Content-Type application/json. Không dùng setTimeout để tự tuyên bố đơn đã hoàn tất.

Server dùng Pydantic giới hạn độ dài chuỗi, số dòng giỏ và số lượng; validation nghiệp vụ sâu tiếp tục ở store.quote() và create_order(). Frontend kiểm tra form để thuận tiện; server vẫn phải kiểm tra vì người dùng có thể gọi API trực tiếp. esc() trong JS và escape() khi viết PDF biến ký tự đặc biệt thành dữ liệu để tên khách không bị hiểu thành HTML.

Đoạn quyết định mã HTTP ở api.py:

```
@router.post('/checkout',status_code=202)
def checkout(req:Checkout):
    # create_order lưu đơn và outbox rồi mới trả kết quả
    oid,created=store.create_order(...)
```

Đây là trích rút gọn để giải thích; dấu ... là phần tham số đã lược, không phải hướng dẫn thay code thật bằng dấu ba chấm. Trong file gốc, decorator và lời gọi đầy đủ nằm trong hàm checkout.

Vì sao 202: đã tiếp nhận để xử lý tiếp, không xác nhận PDF/email xong. Cơ sở là commit đơn + outbox, không phải chờ broker ở trong request. Broker mất kết nối vẫn có thể nhận đơn và giữ outbox pending. HTTP 422 báo dữ liệu không hợp lệ; 404 khi chưa có tài nguyên/file; 409 khi hành động xung đột như thêm quá số worker hoặc reset khi worker còn chạy; 503 ở replay khi Kafka không đọc được.

Nếu xóa/thay: đổi 202 thành 200 chỉ đổi hợp đồng HTTP, không tự làm code đồng bộ. Xóa idempotency key và chỉ giữ khóa nút không ngăn request bị gửi lại do mất kết nối. Dùng FastAPI BackgroundTasks thay cho outbox có thể chạy nền nhưng không tự giữ việc bền khi process chết. Chạy PDF/SMTP ngay trong route làm khách phải chờ và làm lỗi email ảnh hưởng response đặt hàng.

Câu trả lời GV: “202 của em nghĩa là đơn và outbox đã commit. Broker confirm, PDF và SMTP là các mốc sau đó; em không dùng 202 để khẳng định đã gửi email.”

5. Server tính tiền ở đâu và vì sao không tin giá trên trình duyệt?

Mở file: shop/store.py, CATALOG, PRODUCTS, quote(); shop/api.py, Item.

JS tính tạm tổng để khách nhìn thấy, nhưng server chỉ dùng SKU, size và quantity để tra catalogue của mình. quote() xác minh sản phẩm, kích cỡ, số lượng và từ chối hai dòng trùng cùng SKU/size; nhân unit_price với quantity, cộng subtotal, rồi tính shipping. Mốc miễn phí giao hàng là 1.000.000 đ, còn lại 25.000 đ.

Tiền lưu bằng số nguyên VND. Không dùng float để cộng tiền rồi xử lý sai số nhị phân. unit_price, line_total, tên và màu được chụp lại vào đơn khi đặt; catalogue đổi sau đó không làm hóa đơn cũ đổi tiền. Đây là snapshot giá, không phải luôn tra lại giá mới lúc worker tạo PDF.

Ví dụ: 2 áo thun giá 249.000 đ và 1 hoodie giá 549.000 đ: subtotal 1.047.000 đ, shipping 0, total 1.047.000 đ. Client cố gửi unit_price=1 không được dùng để tính tiền. Kiểm thử thực tế dùng đúng tình huống này.

Nếu xóa/thay: tin total/price do JS gửi thì ai gọi API cũng có thể sửa giá. Xóa snapshot rồi đọc catalogue ở worker khiến cùng đơn có thể bị tính theo giá mới. Thay bằng DB sản phẩm là hợp lý khi nhiều sản phẩm hoặc có quản trị giá; nguyên tắc kiểm giá ở server và lưu snapshot vẫn giữ. Sổ tồn kho/thanh toán thật là nghiệp vụ bổ sung chưa có trong demo này.

Câu trả lời GV: “Frontend chỉ dự tính để hiển thị. Số tiền chính thức do server tính từ catalogue và được lưu theo đơn bằng số nguyên VND.”

6. SQLite đang giữ gì? Transaction có tác dụng gì?

Mở file: shop/store.py, init(), connect(), create_order() và commit_invoice().

orders là đơn và snapshot dòng hàng. tasks là trạng thái từng việc invoice/email và lượt nhận. outbox là các yêu cầu publish. invoices là số hóa đơn, tên file và snapshot; PDF thực nằm trên đĩa. deliveries là sổ ứng dụng đã ghi nhận SMTP nhận thư. events ghi dấu vết; workers là quan sát process/assignment; settings giữ nhịp quan sát và Kafka generation.

connect() là context manager. with db: commit khi khối chạy thành công, rollback khi có exception; finally đóng connection. row_factory giúp đọc cột theo tên. busy_timeout cho SQLite chờ khóa thay vì lỗi ngay; WAL hỗ trợ đọc trong khi có ghi, nhưng SQLite vẫn chỉ có một writer tại một thời điểm. Nhiều worker không đồng nghĩa DB ghi vô hạn song song.

create_order() dùng BEGIN IMMEDIATE, kiểm tra request_key cũ và fingerprint. Key cũ cùng nội dung trả cùng order_id; cùng key nhưng nội dung khác bị từ chối. UUID sinh mã có xác suất trùng cực thấp; constraint UNIQUE của DB mới là lớp bảo vệ cuối, không phải tin tuyệt đối vào chuỗi random.

Hai nhóm nguyên tử quan trọng: (1) order + invoice task + outbox invoice; (2) invoice + email task + outbox email. Nếu ghi invoice thành công nhưng ghi outbox email lỗi thì transaction thứ hai rollback, không để hóa đơn có trong DB mà mất yêu cầu email. Unit test cố gây lỗi enqueue để kiểm tra rollback này.

Nếu xóa/thay: bỏ transaction thì có thể lưu nửa chừng. Chỉ kiểm tra “đã tồn tại chưa” trong Python mà bỏ UNIQUE vẫn có race khi hai process chạy cùng lúc. Xóa DB làm mất sổ đơn, task, dedup và liên kết tới PDF; queue còn tin nhưng không còn dữ liệu đơn để xử lý. Thay SQLite bằng PostgreSQL được khi cần nhiều writer/triển khai tách máy; phải mang cả schema, unique key và transaction sang, không chỉ sửa chuỗi kết nối.

Câu trả lời GV: “Em dùng SQLite để cả lớp chạy gọn. Em vẫn dùng constraint và transaction để minh họa tính đúng; đây chưa phải phép thử DB chịu tải thương mại.”

7. Outbox: vì sao không publish ngay sau INSERT là xong?

Mở file: shop/store.py, enqueue/pending_outbox/published/publish_error; shop/api.py, dispatch/startup/shutdown.

Hãy xét code ngây thơ: lưu đơn rồi publish. Nếu process chết đúng khoảng giữa, đơn tồn tại nhưng không có message. Đảo lại publish rồi lưu đơn cũng có khoảng lỗi: worker nhận message nhưng không tìm thấy đơn. Outbox lưu đơn và yêu cầu gửi trong một transaction DB nên cả hai cùng có hoặc cùng không có.

Dispatcher đọc các dòng pending. RabbitMQ: mở connection/channel, khai báo topology, bật confirm_delivery, publish mandatory/persistent và chỉ gọi store.published() khi chặng publish thành công. Kafka: đợi callback delivery/flush thành công rồi mới đánh dấu. Có lỗi thì ghi số lần thử và quay

lại sau; không xóa outbox chỉ vì mất kết nối.

```
commit đơn + outbox
   -> publish -> broker xác nhận -> đánh dấu published trong DB
```

Khoảng giữa broker xác nhận và đánh dấu DB vẫn tồn tại. Process chết ở đó có thể publish lại cùng task; vì vậy outbox giảm nguy cơ bỏ quên việc, không bảo đảm broker chỉ nhận một lần. Consumer phải xử lý lặp an toàn. Một vòng dispatcher riêng cho mỗi transport giúp Kafka lỗi không làm hàng RabbitMQ chờ phía sau Kafka trong cùng vòng.

asyncio.to_thread() cho vòng lặp đồng bộ chạy ở thread khác để không giữ event loop phục vụ HTTP. STOP.wait(.6) vừa giãn lần quét vừa có thể dừng sớm khi shutdown. Đây là polling outbox đơn giản cho một web process; không có cơ chế claim để nhiều bản dispatcher cùng tranh việc. Không chạy uvicorn nhiều worker với thiết kế này.

Nếu xóa/thay: bỏ outbox thì demo happy path vẫn có thể chạy nhưng mất câu trả lời đáng tin cho lỗi giữa DB và publish. Đổi sang Redis list không tự làm INSERT DB và đẩy Redis nguyên tử. CDC/outbox relay riêng là cách triển khai khác khi lớn hơn; vẫn phải xử lý publish lặp. Không cần thêm CDC vào bài seminar này.

Câu trả lời GV: “Outbox bảo vệ ý định phải gửi cùng với đơn, chứ không biến DB và broker thành một transaction. Em chấp nhận giao lặp và chống tạo thêm hóa đơn ở consumer.”

8. RabbitMQ: message được đưa vào đúng queue bằng cách nào?

Mở file: shop/broker.py, connection/topology/rabbit_publish/queue/dlq.

connection() đọc host/port/credential từ environment và tạo Pika BlockingConnection. Heartbeat phát hiện kết nối hỏng; socket timeout và blocked connection timeout giới hạn một số tình huống chờ. Không chia sẻ connection đó tùy tiện giữa các thread. Connection là đường kết nối; channel là kênh AMQP bên trong đường đó.

topology() khai báo direct exchange seminar.shop.v4.tasks. Hai queue là .invoice và .email, binding tương ứng routing key invoice và email. Mỗi queue có .failed riêng và được nối với DLX seminar.shop.v4.errors. Tên queue là địa chỉ công việc; tên consumer là process nhận việc, hai thứ không phải một.

```
exchange tasks -- key invoice --> queue invoice --> consumer tạo PDF
               -- key email   --> queue email   --> consumer gửi SMTP
```

exchange_type='direct' so khớp routing key với binding key. Đây không phải fanout. Muốn hai hệ thống đều nhận một bản sao thì cần queue riêng cho mỗi bên và binding phù hợp; thêm hai consumer vào cùng một queue là chia việc, không phải broadcast.

durable=True giữ định nghĩa exchange/queue qua restart; delivery_mode=2 đánh dấu message persistent. Hai lựa chọn khác nhau và chưa đủ cho câu “không bao giờ mất tin”. mandatory=True giúp publisher nhận biết trường hợp không route đến queue; confirm_delivery giúp nhận kết quả chặng publish. message_id giữ task ID để đối chiếu, không tự khiến RabbitMQ bỏ trùng.

Queue chính có max-length 10.000 và overflow reject-publish. Nếu queue đầy và broker từ chối, dispatcher không được coi việc đã publish thành công; outbox còn pending để xử lý lại. Mốc 10.000 là giới hạn tài nguyên demo, không phải dung lượng tối đa của RabbitMQ nói chung.

Nếu xóa/thay: bỏ binding hoặc đổi routing key sai làm tin không vào đúng queue. Bỏ mandatory có thể khiến publisher không phát hiện tin không route được. Bỏ persistent/durable giảm bảo đảm khi restart.

Đổi cấu hình một queue đã tồn tại mà không đồng bộ định nghĩa có thể gây PRECONDITION_FAILED; đừng sửa tên/loại queue ngay trước buổi trình bày. Hệ thống thật nên cân nhắc quorum queues và cấu hình DLX bằng policy; classic queue một node của bài này không có HA.

Câu trả lời GV: “Exchange route yêu cầu theo role; queue giữ backlog. Em dùng direct vì invoice/email là hai loại công việc xác định, không cần tự sáng tạo cách route trong frontend.”

9. Nút thêm worker có mở chương trình thật không?

Mở file: shop/api.py, add_worker/crash_worker; shop/worker.py, phần if __name__=='__main__'.

Nút gọi POST workers với transport và role. Server dùng subprocess.Popen và cùng Python interpreter để chạy lệnh dưới đây; stdout/stderr ghi vào file log trong data/:

```
python -u -m shop.worker R-HD-xxxxx rabbitmq invoice
python -u -m shop.worker R-MAIL-xxxxx rabbitmq email
```

-m shop.worker chạy module trong package từ thư mục dự án. -u giảm đệm output để log xuất hiện kịp quan sát. CLI phân tích tên/transport/role rồi chọn rabbit() hoặc kafka(). Worker hóa đơn và worker email chạy cùng file nhưng tham số role khác; không cần hai bản code gần giống nhau.

API lưu Popen trong PROCESSES và metadata transport/role trong bộ nhớ để biết process nào nó quản lý. PID trên thẻ lấy từ process thật; p.poll() kiểm tra còn sống. Dòng worker trong DB là quan sát gần nhất, không tự chứng minh process còn sống nếu không đối chiếu. p.kill() dừng thật, chưa phải shutdown mềm chờ xử lý xong.

Giới hạn bốn worker mỗi role/transport tránh bấm quá nhiều trên laptop. Đây là giới hạn UI/API của demo, không phải RabbitMQ chỉ cho bốn consumer. Nếu web restart, các worker do nó quản lý được dừng và không tự mở lại; thêm worker mới để chạy backlog. Process ngoài API không được vẽ là process sống do API quản lý.

Nếu xóa/thay: bỏ Popen và chỉ tạo thêm thẻ HTML thì không còn minh chứng nhiều consumer. Dùng thread để làm worker cũng có thể nhận broker nhưng không chứng minh kill process riêng theo cách hiện tại. Triển khai thật thường dùng worker service/Compose replicas hoặc trình quản lý process; không nên cho người dùng công khai nút mở/kill process không xác thực.

Câu trả lời GV: “Mỗi consumer trong demo là process có PID. Em chọn Popen để lớp điều khiển ngay trên web; khi triển khai tách dịch vụ sẽ để hệ thống vận hành quản lý worker.”

10. Tạo hóa đơn: code nào tạo file và code nào ghi DB?

Mở file: shop/fulfillment.py, fulfill/render_invoice; shop/store.py, invoice/commit_invoice; shop/api.py, invoice_pdf.

Nhánh invoice lấy đơn đã snapshot từ DB. Nếu đã có invoice, nó ghi DEDUP và bỏ qua tạo thêm tác vụ email; nếu file bị thiếu thì có thể tạo lại file từ thông tin hiện có. Nếu chưa có, worker ghi PDF_RENDER, giãn theo setting, đặt số hóa đơn rồi gọi render_invoice.

ReportLab xây tài liệu A4 bằng Paragraph, Table, Spacer. Mỗi dòng sản phẩm có tên/màu/size, quantity, unit_price và line_total; phía dưới có subtotal, shipping và total. Font DejaVu Sans được nhúng để người khác tải PDF vẫn đọc tiếng Việt. escape() ngăn tên/address bị hiểu thành cú pháp markup của Paragraph.

File được ghi vào tên tạm riêng rồi os.replace() thành data/invoices/<order_id>.pdf. Việc thay file cuối tránh đưa ra một file PDF đang ghi dở. Nó không đồng nghĩa file và DB đã commit chung hoặc an toàn trước mọi tình huống mất điện/đĩa. Sau khi tạo file, commit_invoice() mới chốt invoice cùng outbox email.

order_id là primary key của invoices và number có unique constraint. Hai process cùng tạo file cho một đơn có thể làm công thừa, nhưng chỉ lần INSERT invoice thành công mới thêm outbox email. Đừng nói "worker không bao giờ tạo file hai lần"; điều được bảo vệ là bản ghi invoice và yêu cầu email theo đơn.

Modal xem hóa đơn trên web được JS dựng từ snapshot invoice. Nút tải PDF gọi FileResponse lấy file thật; modal không phải nguồn tạo file. Số hóa đơn minh họa có ngày và phần đuôi mã đơn; đây không phải tích hợp hệ thống hóa đơn điện tử thuế.

Nếu xóa/thay: bỏ unique order_id thì redelivery có thể tạo thêm invoice. Ghi email outbox trước khi file sẵn sàng có thể khiến worker email nhận việc nhưng thiếu attachment. Dùng HTML làm hóa đơn được nếu người dùng chỉ cần xem/in HTML; muốn tiếp tục đính kèm PDF phải có bước chuyển đổi phù hợp. Đổi file local thành object storage hợp lý khi worker chạy nhiều máy; khi đó message hoặc DB phải tham chiếu tài nguyên mà các worker đều truy cập được.

Câu trả lời GV: "File nằm trong data/invoices; bản ghi và snapshot nằm trong SQLite. Em không nhầm hiển thị modal với tạo PDF. Hai tài nguyên file/DB có các khoảng lỗi được xử lý bằng retry và idempotency, không phải transaction phân tán."

11. Email có thật không, nhận ở đâu và giới hạn chống trùng là gì?

Mở file: shop/fulfillment.py, nhánh email trong fulfill; shop/store.py, sent/record_sent; service mailpit trong Compose.

Worker kiểm tra sổ gửi, email hợp lệ, bản ghi invoice và file có tồn tại. EmailMessage tạo From, To, Subject, Date, Message-ID và nội dung. add_attachment đọc byte PDF, khai báo application/pdf, đặt filename theo số hóa đơn. smtplib.SMTP nối tới Mailpit; send_message gửi SMTP, không dùng API giả tăng bộ đếm.

Mailpit nhận và giữ email trên máy. Bạn nhập địa chỉ thử bất kỳ hợp lệ nhưng mở thư tại cổng 8025, không chờ Gmail thật. Điều này tránh cấu hình tài khoản/mật khẩu gửi thư cho cả lớp và vẫn kiểm chứng MIME/attachment. SMTP accepted là Mailpit nhận thư, không chứng minh hệ thống Internet hoặc người nhận cuối đã nhận/đọc.

Sau khi SMTP nhận thành công, code mới gọi record_sent và sau đó ACK/commit. Nếu process chết đúng sau SMTP nhưng trước record_sent, lượt sau chưa thấy sổ gửi nên có thể gửi lại. Hai worker gửi cùng đơn đồng thời cũng không được bảo vệ trọn vẹn bằng việc check sổ rồi gửi. Primary key chỉ ngăn thêm hai dòng deliveries, không thu hồi được thư đã đi.

Message-ID ổn định giúp đối chiếu cùng đơn; không có bảo đảm mọi SMTP/mail client sẽ tự loại trùng. Test bình thường chứng minh một tác vụ invoice lặp không thêm invoice/email outbox, chứ không chứng minh exactly-once cho mọi crash window của SMTP.

Nếu xóa/thay: bỏ record_sent thì redelivery email bình thường sẽ gửi lại nhiều hơn. Ghi sổ gửi trước SMTP để tránh trùng có rủi ro khác: đánh dấu đã gửi rồi chết trước send_message khiến mất thư. Muốn đổi sang email Internet phải chọn SMTP thật, credential/secret, TLS và cấu hình gửi phù hợp; không chỉ đổi link Mailpit. Khi cần chặt chẽ hơn, cân nhắc dịch vụ gửi có khóa idempotency và đối soát, vẫn phải hiểu phạm vi bảo đảm của nhà cung cấp.

Câu trả lời GV: “Em gửi SMTP thật vào Mailpit có PDF. Sổ gửi giúp bỏ qua những lần đã ghi thành công, nhưng em không hứa exactly-once qua ranh giới SMTP và SQLite.”

12. ACK, prefetch và crash: cần hiểu ba thời điểm

Mở file: shop/worker.py, rabbit/complete; shop/store.py, received; shop/fulfillment.py, nhánh crash_after_commit.

Worker đăng ký auto_ack=False, prefetch_count=1. Callback giải mã JSON, xác minh task thuộc order/role/transport hiện tại, ghi lượt nhận rồi gọi fulfill. Thành công mới channel.basic_ack(method.delivery_tag); sau đó log ACK_SENT. delivery_tag là số lượt giao trong channel hiện tại, không dùng làm khóa hóa đơn lâu dài.

Thời điểm A, trước commit invoice: worker chết khi đang làm, chưa ACK; broker giao lại, worker khác làm từ đầu. B, sau commit nhưng trước ACK: invoice đã có, delivery vẫn có thể được giao lại; lần sau DEDUP, không thêm invoice/outbox email. C, sau gửi ACK: broker có thể đã xử lý hoặc client mất kết nối trước khi chắc chắn; ACK không có biên nhận ACK ngược cho từng lời gọi như publisher confirm.

Nút sập sau lưu hóa đơn tạo flag cho đơn. Sau commit, os._exit(42) kết thúc process ngay để cố ý không chạy bước ACK. Nếu chỉ có một worker thì cần thêm worker mới; có nhiều worker thì worker còn sống nhận lại. Cờ method.redelivered do AMQP cấp giúp ghi REDELIVER; DEDUP là logic DB của ứng dụng, không phải tên một lệnh broker.

prefetch=1 giúp worker bận chỉ giữ một delivery chưa ACK, consumer khác còn cơ hội nhận việc. Không bảo đảm phân phối thời gian hoàn tất tuyệt đối công bằng; xử lý nhiều consumer có thể hoàn thành khác thứ tự. Đổi prefetch=10 có thể tăng số tin mỗi consumer giữ và ảnh hưởng cách chia backlog; bỏ giới hạn không tự làm CPU nhanh hơn.

Nếu xóa/thay: auto_ack=True xác nhận khi giao theo chế độ đó, worker chết trong nghiệp vụ có thể không được giao lại việc chưa xong. ACK trước fulfill cũng có rủi ro mất việc tương tự. Không ACK và không NACK thì delivery giữ Unacked, với prefetch=1 consumer không nhận tiếp cho tới khi xử lý/xuất hiện mất kết nối. Xóa dedup làm side effect lặp có thể thành thêm hóa đơn. requeue=True liên tục với tin không thể sửa sẽ tạo vòng lặp nóng.

Câu trả lời GV: “Em ACK sau xử lý để broker có thể giao lại khi process chết; vì giao lại có thể xảy ra sau commit nên em cần idempotency. ACK và dedup giải quyết hai vấn đề khác nhau.”

13. Lỗi email, DLQ và retry đi qua những hàm nào?

Mở file: shop/worker.py, nhánh except của callback Rabbit; shop/api.py, fix_email/retry_task; shop/broker.py, DLX bindings.

Checkout bình thường chặn email sai. Seed quản trị cố ý tạo email không hợp lệ để mô phỏng dữ liệu lỗi từ kênh tích hợp khác, không phải bảo checkout luôn cho nhập sai. Cả RabbitMQ và Kafka cùng gặp lỗi từ hàm fulfillment đó.

Ở RabbitMQ, lỗi xử lý ghi NACK_DLQ và gọi basic_nack với requeue=False. Nếu có DLX/binding đúng, broker route message vào queue failed. Consumer tiếp tục nhận tin khác; PDF của đơn lỗi email vẫn giữ vì đó là bước trước đã hoàn thành.

Nút sửa gửi PATCH email rồi POST retry. API lấy đúng message của task từ DLQ, publish lại vào queue chính có confirm, sau đó ACK bản đang giữ từ DLQ. Tin đã xem nhưng không phải task cần retry chưa ACK nên sẽ được requeue khi connection đóng. Code duyệt tối đa 200 bản; đây là cách gọn cho demo,

không phải công cụ tìm kiếm DLQ lớn. Nếu không thấy, API trả lỗi để người dùng chờ/kiểm tra thay vì giả báo đã thử lại.

Tất cả exception trong xử lý Rabbit hiện đều đi nhánh DLQ, kể cả lỗi tạm thời của SMTP/file/DB. Demo chưa phân loại retry tạm thời tự động theo backoff. Sau khi dịch vụ hồi phục, người vận hành retry; với lỗi PDF có thể POST task invoice retry qua API, trường sửa email trên UI chủ yếu phục vụ tình huống lỗi email. Khi triển khai thật cần phân loại transient/permanent và tránh yêu cầu sửa email cho lỗi đĩa.

Nếu xóa/thay: bỏ DLX rồi vẫn NACK requeue=False có thể loại message khỏi queue mà không giữ chỗ để sửa. ACK bản DLQ trước khi publish lại có nguy cơ mất việc nếu publish thất bại. Publish trước ACK giúp giảm mất việc nhưng vẫn có thể trùng nếu chết ở giữa; giữ task ID và dedup là cần thiết. Tự động requeue ngay tin sai email không sửa được email.

Câu trả lời GV: “DLQ là nơi giữ công việc không xử lý được để quan sát, sửa và đưa lại. Nó không tự sửa lỗi; app của em retry sau khi đã có quyết định hoặc dữ liệu đúng.”

14. Kafka: khác ở transport, không đổi nghiệp vụ để thiên vị

Mở file: shop/broker.py, ensure_kafka/kafka_publish; shop/worker.py, kafka; shop/api.py, analytics_replay.

Kafka tạo topic invoice/email theo generation phiên, hai partition cho mỗi topic công việc và một partition cho mỗi topic dead. Key là order_id. Producer dùng enable.idempotence và acks=all, đợi callback delivery. RF=1 nên acks=all ở đây không tương đương nhiều bản sao hoặc HA. Producer idempotence không chống mọi lần publish lặp do DB outbox hoặc producer mới tạo ở lần sau.

Consumer dùng classic group, group ID riêng theo topic/role, auto commit tắt. on_assign/on_revoke nhận assignment thật và lưu để UI hiển thị. Với hai partition, tối đa hai consumer trong cùng group được gán partition tại một thời điểm. Bốn process vẫn có thể tồn tại nhưng hai process rảnh; thêm consumer không tự tạo partition.

Thành công gọi cùng fulfill rồi commit offset đồng bộ. Commit một record thường lưu vị trí kế tiếp để lần đọc sau bắt đầu sau record đã hoàn thành; không xóa record khỏi log. earliest chỉ có tác dụng khi group chưa có offset hợp lệ cho partition hoặc cần reset theo cấu hình, không có nghĩa mỗi lần poll đều đọc từ đầu.

Lỗi nghiệp vụ làm pause đúng partition và giữ record chưa commit trong bộ nhớ worker. Consumer vẫn poll để duy trì hoạt động group; partition khác được giao vẫn có thể chạy. Seed lỗi đặt ba đơn cùng partition 0 có chủ ý để dễ nhìn chính sách giữ thứ tự; không đại diện mọi cách phân bố thực tế.

Sửa/retry xử lý lại record lỗi. Chuyển topic lỗi tạo envelope chứa source topic/partition/offset, publish và đợi delivery rồi commit nguồn, resume partition. Kafka không bắt buộc có chính sách pause đó; app chọn giữ thứ tự. Khi crash giữa publish lỗi và commit nguồn có thể thêm bản lỗi lặp. Kafka transactions có thể hỗ trợ một số luồng Kafka→Kafka nhưng không tự bao trùm file, SQLite và SMTP.

Replay tạo group analytics mới đọc log retained hiện tại, trả order_id/partition/offset và không gọi fulfill. Record vẫn có sau commit group xử lý nên có dữ liệu đọc lại; giới hạn 100 record và một khoảng chờ để phù hợp seminar. Đây là đọc lại task log, chưa phải một mô hình phân tích doanh thu đầy đủ.

Nếu xóa/thay: bật auto commit có thể ghi tiến độ trước nghiệp vụ xong. Commit trước fulfill có rủi ro bỏ việc sau crash; không commit có thể gặp lại record. Tăng partition có thể tăng số consumer có assignment nhưng cần xét key/order và phân bố. Retry topic/DLQ hoặc xử lý song song trong consumer là cách khác; cần tự thiết kế offset và thứ tự. Share Groups là mô hình khác ngoài classic

group đang chạy. RabbitMQ Streams cũng có log/replay, nên không nói RabbitMQ nói chung không thể replay.

Câu trả lời GV: “Em đang so work queue với Kafka classic group hai partition. RabbitMQ thuận tiện cho công việc độc lập; Kafka thuận tiện giữ lịch sử cho group khác. Em cho thấy cả cách gỡ lỗi Kafka và lợi thế replay, không kết luận từ đồng hồ trên laptop rằng một broker luôn nhanh hơn.”

15. Docker và từng thư viện đảm nhận phần nào?

docker-compose.yml mở web, rabbitmq, mailpit và Kafka khi bật profile kafka. Host rabbitmq, mailpit, kafka là tên service trong network Docker. Trong container web, localhost là chính container web, không phải container RabbitMQ; vì thế không tự đổi host bên trong thành localhost.

127.0.0.1:8000:8000 bind web trên máy hiện tại. Port đầu là cổng máy bạn, port sau là cổng trong container. Đổi WEB_PORT đổi URL mở trên máy, không đổi cổng Python bên trong. RabbitMQ có AMQP 5672 để code nói chuyện và Management 15672 cho UI/API quản trị; nhầm hai cổng sẽ không kết nối đúng giao thức.

./data:/lab/data là bind mount giữ DB/PDF/log ngoài container. rabbit-data, kafka-data, mail-data là named volumes. docker compose down giữ chúng; down -v là xóa volume, không dùng khi cần giữ dữ liệu. Khi sửa code phải build lại image để container nhận code mới vì source không được mount live.

Dockerfile cài requirements rồi copy app/shop/static. Thư mục docs/tests/data/.env không được đưa vào image qua build context theo dockerignore; file data được mount lúc chạy. Requirements cố định phiên bản để lớp dùng cùng bộ thư viện, không khẳng định là các bản mới nhất. .env.example chứa cấu hình demo; không thay bằng token/mật khẩu thật rồi commit.

FastAPI nhận HTTP/route/lifespan; Uvicorn chạy server ASGI; Pydantic đọc và kiểm cấu trúc dữ liệu. Pika nói AMQP với RabbitMQ. confluent-kafka nói Kafka với producer/admin/consumer. ReportLab tạo PDF. sqlite3, smtplib, subprocess, asyncio, uuid, json và pathlib là thư viện chuẩn Python. Frontend dùng JavaScript và CSS thuần, không cần Node khi người học chạy Docker.

Nếu xóa/thay: bỏ font tiếng Việt có thể tạo PDF thiếu dấu/lỗi font. Bỏ volume data làm dữ liệu gắn với container và dễ mất khi recreate. Có thể dùng framework HTTP khác nhưng phải thay toàn bộ route/validation/lifespan, không có lợi cho mục tiêu seminar hiện tại. Chỉ muốn học RabbitMQ thì không bật Kafka; muốn so sánh phải bật đúng profile.

16. Phần nào có thể bỏ, phần nào không nên xóa khi học?

- Có thể bỏ ảnh đẹp hoặc thay CSS mà giữ nghiệp vụ. Không bỏ id/selector nút trong HTML nếu JS vẫn dùng chúng; JS sẽ không tìm thấy element và có thể dừng khởi tạo.

- Có thể bỏ nhịp quan sát bằng setting 0; không phải xóa worker/queue. Tạo PDF/SMTP vẫn diễn ra.

- Có thể bỏ Kafka khỏi buổi học RabbitMQ bằng cách không bật profile. Không xóa chỉ một nhánh import nhưng giữ nút và test Kafka rồi cho rằng dự án vẫn hoàn chỉnh.

- Có thể bỏ công cụ build PDF tài liệu nếu chỉ chạy demo; mất khả năng tái tạo tài liệu in, không mất invoice vì invoice có render riêng.

- Không bỏ validation giá server, transaction, unique keys, outbox và ACK muộn nếu vẫn muốn bảo vệ các tình huống lỗi đang trình bày.

- Không xóa dữ liệu chỉ vì “không phải code”. DB và volume có thể chứa đơn cần giữ; reset hoặc dọn dữ liệu là hành động riêng.

Các file v3 trùng mục tiêu đã được loại khỏi source: API/worker/producer/SQLite cũ, Kafka replay riêng cũ, legacy HTML/JS/CSS, test v3 và hướng dẫn v3. Chỉ còn bộ API/store/broker/worker/fulfillment của cửa hàng. Phần connection AMQP dùng chung trước đây được chuyển vào broker của cửa hàng, không xóa mất nó. Endpoint /lab và /api/state v3 không còn; dùng /api/shop/state.

Các môi trường cá nhân như .venv/.vscode và dữ liệu đang có không bị tự xóa. Những bản test/cache có thể cất ngoài thư mục dự án sau khi dừng container tương ứng. Giấy phép font phải giữ khi phát hành font.

17. Hai mươi câu hỏi GV hay xoáy và cách trả lời ngắn

1. Sao không viết hết trong một file? “Có thể, nhưng HTTP, lưu trữ, giao tiếp broker, consumer và nghiệp vụ có trách nhiệm khác nhau. Năm module đủ lần theo luồng mà không tạo thêm tầng trừu tượng.”

2. Producer chính xác nằm đâu? “Dispatcher trong api.py trực tiếp publish qua broker.py. Handler checkout tạo order/outbox; việc ghi outbox chưa phải broker đã nhận message.”

3. API có chờ worker không? “Không chờ PDF/email; nó chờ phần validation và transaction DB. HTTP vẫn có xử lý đồng bộ bên trong, nhưng nghiệp vụ phía sau được tách qua queue.”

4. Sao đã có queue vẫn cần DB? “Queue giữ yêu cầu làm việc, DB giữ sự thật nghiệp vụ và trạng thái/idempotency. Message chỉ tham chiếu mã đơn, không phải toàn bộ dữ liệu cửa hàng.”

5. Hai consumer cùng queue có cùng nhận một message không? “Bình thường broker phân phối delivery cho một consumer; nhiều consumer chia việc. Vẫn có thể gặp cùng task ở lần giao lại hoặc khi producer publish lặp.”

6. Em chứng minh async bằng gì? “Đặt hàng lúc 0 worker: nhận 202, DB có đơn/outbox, queue có Ready nhưng chưa có PDF. Bật từng role mới xuất hiện PDF rồi email.”

7. Em chứng minh worker thật bằng gì? “Mỗi worker có PID, log riêng và callback nhận message; kill process làm kết nối broker mất và có giao lại hoặc Kafka rebalance.”

8. Có ACK thì không bao giờ mất/trùng? “Không. ACK chỉ xác nhận một lượt giao. Cần xét thời điểm crash, idempotency, xác nhận publisher, loại queue, replication và dữ liệu bên ngoài broker.”

9. Có thể ACK ở frontend không? “Không dùng luồng đó ở đây. Frontend chỉ gọi HTTP; delivery_tag/channel thuộc consumer đang kết nối broker.”

10. REDELIVER là em tự giả lập chữ? “Với RabbitMQ, code dựa vào method.redelivered từ broker và log lượt nhận. Tình huống crash thật khiến broker giao lại. Với Kafka, chữ đó là log app suy ra task đã từng nhận, không phải cờ AMQP.”

11. DEDUP có phải RabbitMQ tự làm? “Không. Em kiểm bản ghi invoice unique theo order_id và chỉ tạo email outbox ở lần insert invoice thành công.”

12. Tắt email worker có mất đơn không? “Không vì công việc email còn ở queue/outbox theo trạng thái publish. PDF và đơn đã lưu vẫn có; thêm email consumer để tiếp tục.”

13. PDF nằm đâu nếu xóa container? “Với Compose hiện tại, data là bind mount nên còn trên máy host. Xóa thư mục data là xóa dữ liệu khác với chỉ recreate web container.”

14. Khi broker tắt có nhận đơn được không? “Có nếu DB/web còn hoạt động. Transaction lưu order/outbox; dispatcher gửi lại sau. DB hỏng thì không có cơ sở để trả 202 thành công.”

15. Nếu web process chết thì sao? “Đơn/outbox đã commit còn trong data. Khởi động web lại chạy dispatcher, nhưng worker cần được mở lại. Đó là giới hạn quản lý process cục bộ của demo.”

16. DLQ có tự retry theo lịch không? “Không ở bản này. Người vận hành sửa/ra quyết định rồi retry; tự động retry/backoff là phần cần thiết kế thêm khi triển khai.”

17. Kafka không thể retry à? “Có. Demo có retry tại partition hoặc chuyển topic lỗi rồi đi tiếp. Em chỉ ra chi phí thiết kế offset/thứ tự trong mô hình đang chọn.”

18. Thêm hai partition có làm Kafka bằng RabbitMQ không? “Có thể cho nhiều consumer có assignment hơn, nhưng hai hệ vẫn khác mô hình log và work queue. Cần xét key, thứ tự, retention và nhu cầu, không kết luận chung từ số worker.”

19. Demo này đã là hệ thống SOA production chưa? “Chưa. Có tách HTTP, broker và worker nhưng vẫn chia sẻ SQLite/file local cho dễ clone. Chứng minh cơ chế tích hợp bất đồng bộ, không phải hoàn chỉnh ranh giới dịch vụ thương mại.”

20. Em tự hiểu code hay chỉ nhờ AI? “Em có dùng AI hỗ trợ, nhưng em lần được checkout→transaction/outbox→publish→consumer→PDF/SMTP→ACK; em chỉ rõ các điểm lỗi, code bảo vệ và giới hạn còn lại. Em có kiểm thử thực tế thay vì chỉ đọc tên pattern.”

18. Cách tập đọc code trong 40 phút

10 phút đầu: mở sơ đồ, tìm handler checkout trong JS và route checkout trong api.py; đọc create_order đến enqueue. Đặt một đơn lúc 0 worker và đối chiếu DB/outbox, không nhảy ngay sang code Kafka.

10 phút tiếp: mở broker.topology và worker.rabbit. Viết ra ba dòng riêng “publish confirm”, “fulfill”, “consumer ACK”; giải thích bên nào gửi mỗi loại xác nhận. Thêm worker hóa đơn trước, xem PDF; thêm worker email sau, mở Mailpit.

10 phút tiếp: mở commit_invoice và nhánh crash_after_commit. Dự đoán nếu process chết trước/sau commit thì việc gì còn; chạy thí nghiệm để kiểm tra dự đoán. Tập nói giới hạn SMTP trùng bằng chính hai dòng send_message/record_sent.

10 phút cuối: mở Kafka consumer config, on_assign, commit và pause. So sánh cùng 4 worker/2 partition; chạy lỗi rồi bấm chuyển topic lỗi, cuối cùng replay. Chọn câu hỏi ở phần 17 và trả lời không nhìn tài liệu.

Không cần học thuộc import, từng mã màu CSS hoặc tên class của ReportLab. Cần tự chỉ được đúng file/hàm và giải thích thứ tự gây ra kết quả. Khi muốn thử xóa/đổi một cơ chế, dùng nhánh Git hoặc bản clone thử và dữ liệu riêng; đừng phá bản chạy buổi seminar.

19. Nguồn chính thức để kiểm tra lời giải thích

Các diễn giải là tóm lược gắn với code demo, không trích nguyên văn. RabbitMQ /docs có thể mở phiên bản mới hơn image đang chạy; dùng bộ khái niệm đúng loại queue và cấu hình, không tự đổi image trước buổi trình bày.

- RabbitMQ Work Queues: work distribution, ACK, persistence và prefetch.

- Consumer Acknowledgements and Publisher Confirms: hai chặng xác nhận độc lập, requeue và giới hạn.

- RabbitMQ Dead Letter Exchanges: điều kiện dead-letter, binding, policy và an toàn.

- Kafka 4.2 Design: log, partition/group, offset và delivery semantics.

- RabbitMQ Streams: log/replay ngoài classic work queues.

- Mailpit API: xem message, raw MIME và attachment để đối chiếu.

Phụ lục vị trí hàm trong PDF được lấy từ AST của chính source khi tạo tài liệu, tránh học theo số dòng của bản file cũ.

Kịch bản Mộc Studio — 15–20 phút

Chạy docker compose --profile kafka up -d --build --wait --wait-timeout 240. Mở web cổng 8000 và Mailpit cổng 8025 trước giờ trình bày. Bắt đầu với 0 worker, chọn nhịp quan sát 4s. Nếu dùng dữ liệu cũ, dừng worker và reset cửa hàng trước. Không cần quay video.

0–3 phút: vấn đề thực tế và đặt hàng

Nói: “Một trang bán hàng cần ghi nhận đơn nhanh. Tạo PDF và gửi email là việc phía sau; nếu gộp hết vào HTTP request, khách phải chờ cả hai bước. Tôi sẽ đặt một đơn, rồi mở từng loại worker để thấy sự tách biệt.”

Thêm 2 áo thun, nhập thông tin ví dụ, đặt hàng. Tổng 498.000 + giao hàng 25.000 = 523.000 đ. Trang chuyển sang chi tiết đơn. HTTP 202 xuất hiện trong thông báo; có thể mở Network → checkout để chỉ status thật.

Nói: “202 nghĩa là server đã lưu đơn và yêu cầu xử lý vào cùng transaction. Chưa có worker nên chưa có hóa đơn. Dispatcher gửi yêu cầu trong outbox tới broker và chờ confirm; khác consumer ACK sau xử lý.”

3–6 phút: tách hai consumer

Sang Phòng vận hành → RabbitMQ. Chỉ queue hóa đơn có Ready. Thêm một consumer Tạo hóa đơn PDF, xem mã đơn trên thẻ worker và Unacked nếu kịp chu kỳ cập nhật. Sau đó chỉ ra PDF đã có, queue email còn chờ. Mở chi tiết đơn và xem hóa đơn.

Thêm consumer Gửi email kèm PDF, mở Mailpit, chọn thư theo số hóa đơn, tải file đính kèm.

Nói: “Đây là file PDF được tạo từ từng dòng hàng, không phải chỉ đổi trạng thái trên giao diện. Email đi qua SMTP và có file đính kèm. Mailpit giữ thư trên máy để cả lớp chạy không cần tài khoản email. Consumer hóa đơn và consumer email có thể tăng/giảm độc lập.”

6–9 phút: backlog và thêm worker

Đặt nhịp 4s, bấm Tạo 6 đơn hàng. Khi worker đầu đang bận, thêm consumer hóa đơn thứ hai rồi thứ ba. Chỉ các mã đơn khác nhau đang xử lý ở các process/PID khác nhau. Mở payload/outbox để thấy task_id, order_id, role.

Nói: “Queue đệm công việc khi đầu vào lớn hơn năng lực xử lý. prefetch=1 giới hạn mỗi consumer giữ một tác vụ chưa ACK. Chúng ta tăng năng lực tạo PDF mà không đổi API đặt hàng. Bốn giây ở đây là nhịp quan sát có thể tắt, không phải RabbitMQ truyền một tin mất bốn giây.”

9–12 phút: tin lỗi và phục hồi

Bấm Đơn lỗi email + 2 đơn tốt. Chỉ PDF của cả ba đã tạo, một tác vụ email sai ở DLQ, hai đơn tốt nhận thư. Sửa email và retry, mở thư vừa được gửi.

Bấm Sập sau lưu hóa đơn khi có từ 2 consumer hóa đơn; theo mã đơn trong log: INVOICE_COMMIT, CRASH_BEFORE_ACK, REDELIVER, DEDUP, ACK_SENT. Nếu chỉ có một consumer thì thêm consumer mới sau crash.

Nói: “ACK là consumer báo đã xử lý xong lượt giao. Consumer có thể đã lưu hóa đơn nhưng chết trước ACK nên broker giao lại. Mã đơn duy nhất và transaction giúp không tạo thêm hóa đơn. Đây không phải lời hứa exactly-once cho mọi thứ; SMTP và DB vẫn có một khoảng lỗi có thể làm gửi thư lặp.”

12–17 phút: Kafka cùng nghiệp vụ

1. Mở RabbitMQ & Kafka, bấm Chuẩn bị 4 worker hóa đơn mỗi bên và đợi thông báo sẵn sàng; bấm Nạp 6 đơn thật mỗi bên. Quan sát assignment: RabbitMQ có 4 consumer trên queue, Kafka có 4 process nhưng chỉ 2 process được giao partition. 2. Bấm Chạy tình huống lỗi trên cả hai broker. RabbitMQ gửi được hai thư tốt; Kafka giữ record email sai ở partition 0 nên hai record sau trong partition đó chưa gửi. 3. Vào phòng vận hành, chọn Kafka; bấm Chuyển sang topic lỗi. Chỉ KAFKA_APP_DLQ, OFFSET_COMMIT của record tiếp theo, hai email tốt xuất hiện. Có thể sửa và retry đơn đã chuyển lỗi. 4. Quay lại so sánh, bấm Đọc lại bằng group analytics mới. Chỉ group ID, partition, offset; tổng PDF/email không tăng.

Nói: “Với công việc gửi hóa đơn độc lập, RabbitMQ cho cơ chế ACK/NACK từng việc và DLX sẵn trong broker, thêm consumer trên queue thuận tiện. Kafka classic group gắn mức song song với partition. Ứng dụng này giữ thứ tự nên phải quyết định retry hoặc DLQ khi record lỗi. Kafka vẫn làm được; tăng partition hoặc xây retry/DLQ là phần thiết kế thêm. Khi cần lịch sử để ứng dụng khác đọc lại, Kafka lại thuận tiện.”

Không gọi đây là benchmark tốc độ. Hai bên dùng cùng code PDF/SMTP, số partition đã ghi rõ. Kafka Share Groups và RabbitMQ Streams là các mô hình khác ngoài phần đang chạy.

17–20 phút: chốt ý và để lớp thử

Chỉ README trên GitHub, lệnh clone và một lệnh Compose. Mỗi người chạy trên máy mình. Mời người nghe dự đoán: nếu tắt consumer email thì API đặt hàng, PDF và thư thay đổi thế nào? Sau đó thử ngay trên giao diện.

Nếu chỉ có 15 phút, rút phần chia việc còn 1 phút và giữ một tình huống crash; không bỏ bước mở email và file PDF vì đó là kết quả nghiệp vụ cụ thể.

So sánh cho bài toán tạo hóa đơn và gửi email

Đối chiếu tài liệu chính thức ngày 28/10/2026. Demo cố định RabbitMQ 4.3.6 và Kafka 4.2.0; không khẳng định đây là bản mới nhất.

Điều kiện chung và khác biệt có chủ ý

Hai transport cùng catalogue, giá server, SQLite, shop/fulfillment.py: fulfill(), PDF ReportLab và SMTP Mailpit; nhịp quan sát cùng một setting. Mỗi hệ chạy một broker trên cùng máy. RabbitMQ dùng classic work queues, Kafka dùng Consumer Groups truyền thống, mỗi process xử lý tuần tự bản ghi đã poll. Kafka có 2 partition/topic, RF=1; không dùng Share Groups. Đây là so sánh mô hình phục vụ bài toán, không phải benchmark năng lực cực đại.

Bài toán: Thêm 4 worker xử lý các hóa đơn độc lập · RabbitMQ: 4 consumer cùng queue, prefetch=1 · Kafka: 4 consumer trong group, 2 partition → 2 consumer có assignment · Kết luận: RabbitMQ thêm mức song song mà không cần tăng partition cho bài toán này

Bài toán: Một email sai, hai email đúng phía sau · RabbitMQ: NACK riêng tin lỗi, DLX route tới DLQ; tin tốt tiếp tục · Kafka: Cùng partition 0, chính sách giữ thứ tự pause ở lỗi, chưa commit · Kết luận: RabbitMQ có ACK/NACK và dead-letter routing từng tin; Kafka app cần chiến lược offset/retry phù hợp

Bài toán: Khôi phục lỗi Kafka · RabbitMQ: Không áp dụng · Kafka: App publish .email.dead, đợi delivery, commit nguồn rồi resume · Kết luận: Kafka làm được DLQ; trong demo đây là code ứng dụng, không phải Kafka tự tạo DLQ

Bài toán: Sập sau lưu PDF trước xác nhận · RabbitMQ: Unacked được giao lại khi kết nối chết · Kafka: Record chưa commit được group đọc lại sau đổi owner · Kết luận: Cả hai cần idempotency với hiệu ứng bên ngoài broker

Bài toán: Thêm ứng dụng phân tích sau khi đã xử lý · RabbitMQ: Tin trong classic queue đã ACK không còn để consumer mới đọc lại · Kafka: Group analytics mới đọc lịch sử còn retention, không làm lại nghiệp vụ · Kết luận: Kafka log thuận tiện cho nhiều ứng dụng độc lập dùng lại lịch sử

Thí nghiệm 1: partition là đơn vị gán việc của classic group

shop/broker.py: PARTITIONS=2; mỗi vai trò có topic riêng. shop/worker.py: kafka() đăng ký callback on_assign, lưu assignment thực vào SQLite. UI đếm consumer có partition, không đếm process đang chạy thành năng lực xử lý. Nút chuẩn bị chờ 4 RabbitMQ consumer và 2 Kafka consumer có assignment trước khi báo sẵn sàng.

Có thể tăng partition để có 4 consumer hoạt động, hoặc lập lịch song song trong consumer; cần thiết kế xử lý offset khi hoàn tất lệch thứ tự và bảo vệ thứ tự theo key. Tăng partition cũng có ảnh hưởng phân bố key. Không diễn giải kết quả là Kafka chỉ chạy được hai worker hoặc RabbitMQ luôn nhanh hơn.

Thí nghiệm 2: lỗi nghiệp vụ khác lỗi broker

Seed tạo email thiếu dấu @ để handler thật từ chối; không ném lỗi riêng chỉ bên Kafka. Cùng ba đơn được route partition 0 trong thí nghiệm để dễ nhìn thấy thứ tự. Kafka callback giữ message lỗi trong bộ nhớ và pause đúng partition đó; vẫn poll giữ group membership. Partition khác không bị pause bởi quy tắc này.

Ứng dụng có hai cách khôi phục: sửa địa chỉ rồi xử lý lại record chưa commit, hoặc publish error envelope vào topic .dead, đợi xác nhận rồi commit nguồn và resume. Crash giữa publish DLQ và commit có thể sinh bản lỗi lặp. Kafka transactions có thể giải quyết một số luồng Kafka→Kafka; không tự làm SQLite/SMTP thành transaction Kafka.

RabbitMQ khai báo DLX và binding tới .failed; basic_nack(requeue=False) kích hoạt routing lỗi. Với classic queue một node, đây không phải bảo đảm tuyệt đối chặng DLQ không mất tin khi đích lỗi. Có thể chọn quorum queues và chính sách phù hợp khi triển khai thật.

Thí nghiệm 3: lợi thế log của Kafka

POST /api/shop/replay tạo group analytics ID mới, đọc topic hiện tại từ earliest còn retention và commit offset của group riêng. Chỉ đọc tối đa 100 record, không gọi fulfill(). Cửa hàng hiển thị bản ghi công việc (task), không giả vờ đó là hệ thống phân tích doanh thu đầy đủ. Tiền và hóa đơn trong DB không thay đổi vì đọc lại.

Muốn fanout RabbitMQ tới nhiều ứng dụng, có thể route bản sao tới các queue riêng ngay từ đầu. Muốn lưu log/replay theo mô hình RabbitMQ, xem Streams; không nói RabbitMQ nói chung không thể replay. Kafka Share Groups có chia sẻ record và xác nhận record, khác mô hình classic group của thí nghiệm.

Kết luận dùng để thuyết trình

“Trong luồng gửi hóa đơn gồm các công việc độc lập, RabbitMQ cho cách phân phối, xác nhận và cô lập lỗi trực tiếp ở mức message. Kafka mạnh khi coi dữ liệu là log để nhiều ứng dụng đọc và đọc lại. Sự phù hợp phụ thuộc mô hình xử lý, thứ tự, lịch sử và vận hành; không kết luận từ tên sản phẩm hoặc vài giây trên một laptop.”

Nguồn học chính thức

- RabbitMQ Work Queues: chia việc, manual ACK, persistence, prefetch.

- RabbitMQ ACK và publisher confirms: hai chặng xác nhận độc lập, Unacked và requeue khi kết nối mất.

- RabbitMQ Dead Letter Exchanges: điều kiện dead-letter, routing và giới hạn an toàn.

- Apache Kafka 4.2 Introduction: topic, partition, consumer groups và retention.

- Apache Kafka 4.2 Design: assignment theo partition, delivery semantics, offset và Share Groups.

- RabbitMQ Streams: mô hình log/replay khác classic work queue.

- Mailpit: SMTP, giao diện xem thư, attachments và API kiểm thử.

Đánh giá demo sau kiểm thử ngày 31/10/2026

Kết luận theo bằng chứng

Đủ ý nghĩa và trực quan cho seminar về RabbitMQ trong luồng xử lý đơn hàng. Sản phẩm đầu ra là đơn được tính tiền/lưu ở server, PDF có chi tiết và email SMTP có PDF thật. Giá trị queue xuất hiện khi chưa có consumer, có backlog, worker sập hoặc một email sai. Không chỉ là mô phỏng giao diện đổi trạng thái.

Chưa phải chứng minh một cửa hàng production đã hoàn chỉnh. Không có thu tiền, tồn kho, vận chuyển thật, auth, nhiều máy, replication/HA, benchmark tải lớn hoặc bảo đảm không gửi email trùng trong mọi trường hợp. Mailpit là SMTP cục bộ. Phần Kafka cho thấy khác biệt của cấu hình classic group hai partition và chính sách giữ thứ tự của ứng dụng.

Môi trường và cách thực hiện

Kiểm tra bản cửa hàng đã gom vào package shop, dùng Python 3.12 trong Docker, RabbitMQ 4.3.6, Kafka 4.2.0, Mailpit 1.27.4. Dựng Compose project thử riêng moc-final-20261031, web 18000, Mailpit 18025, DB/volume thử riêng; không reset DB đơn hàng của bản chính. Source frontend sau định dạng và backend sau di chuyển đã được build vào image kiểm tra.

Chạy lại sáu unit tests, bài integration với broker thật, kiểm tra nghiệp vụ ở nhịp 0s, bài trình duyệt mua hàng và bài trình duyệt so sánh. Dùng Edge qua Playwright ở 1440×1000 và 390×844. Không quay video.

Những gì đã qua kiểm tra

- PASS - 0 consumer: HTTP 202 thật, giá server 523.000 đ cho hai áo và giao hàng, cùng key trả cùng đơn, queue Ready=1, chưa có invoice.

- PASS - hai vai trò tách biệt: consumer invoice tạo PDF khi email consumer chưa mở; thêm email consumer mới có SMTP/email.

- PASS - PDF và attachment: có file PDF thật, email có application/pdf lớn hơn 1 KB. Unit test trích được tên tiếng Việt, tên sản phẩm và tổng tiền trong PDF.

- PASS - task trùng: gửi lại task invoice cùng mã, có DEDUP, không tăng số invoice/email đã ghi.

- PASS - cùng nghiệp vụ hai broker: 6 đơn mỗi bên, tổng 12 đơn tạo PDF và email qua cùng fulfillment.

- PASS - chia việc: RabbitMQ có 4 consumer hóa đơn; Kafka 4 consumer nhưng chỉ 2 consumer có assignment trên topic hai partition, 2 consumer unassigned.

- PASS - email lỗi: RabbitMQ cô lập một tin lỗi và gửi hai thư tốt; Kafka cùng partition 0 pause theo chính sách hiện tại, hai thư tốt chưa đi tiếp trước quyết định xử lý.

- PASS - Kafka DLQ thật: chuyển lỗi publish được record trong topic dead; group mới đọc thấy record đó; hai email tốt phía sau tiếp tục.

- PASS - sửa và retry: giữ mã đơn/task, sửa email và gửi được thư ở cả hai transport.

- PASS - crash sau commit trước ACK/offset: cả hai transport nhận lại invoice task, có DEDUP và nhiều lượt nhận nhưng không tạo thêm invoice.

- PASS - analytics: group mới đọc log thực và không làm tăng invoice/email.

- PASS - RabbitMQ tạm dừng: checkout vẫn lưu order/outbox; sau khi broker trở lại, tác vụ tiếp tục và đơn hoàn tất.

- PASS - nghiệp vụ nhịp 0s: hai dòng sản phẩm, tổng 1.047.000 đ, miễn giao hàng; byte PDF lấy từ MIME email có SHA-256 giống PDF tải qua API.

- PASS - dữ liệu không hợp lệ: email sai, size không tồn tại và dùng key cũ với nội dung khác bị HTTP 422; giá client gửi không được dùng tính tiền.

- PASS - giao diện: đặt đơn bằng form, 202, mở đúng PDF của đơn vừa đặt; bốn tab không tràn ngang ở 390px, không có lỗi JavaScript pageerror trong bài kiểm tra.

- PASS - nút so sánh: chuẩn bị worker, nạp hai lô 6 đơn, quan sát 2/3 và 0/3 thư, chuyển lỗi rồi cả hai bên 2/3; replay hiển thị partition/offset thật.

- PASS - dọn lab: đường dẫn /lab, /api/state cũ và tài nguyên legacy trả 404; luồng /api/shop tiếp tục chạy sau khi chuyển package.

Sáu unit tests còn kiểm tra: request đồng thời cùng key chỉ tạo một đơn; rollback invoice/email outbox cùng nhau; backlog Kafka không chặn truy vấn outbox RabbitMQ; assignment Kafka được giữ khi worker đổi trạng thái đang làm việc.

Một sai lệch của kiểm thử đã được sửa

Lần kiểm tra ban đầu dừng ở phép so sánh tổng số email trước/sau task trùng. Snapshot lấy counters và chi tiết đơn bằng các lần đọc riêng nên có thể thấy order completed trong khi counters ở lần đọc trước vẫn là 0. Quan sát sau đó có đúng một invoice/một email, không phải đã tạo hai email. Điều kiện test được đổi thành chờ cả trạng thái và counters ổn định trước khi lấy mốc so sánh; thêm guard chỉ chạy integration trên DB thử trống. Bài integration sau sửa chạy qua toàn bộ. Đây không phải kiểm thử chứng minh snapshot có tính nguyên tử.

Cách trình bày để người nghe thấy ý nghĩa

1. Bắt đầu với 0 worker, đặt hàng qua cửa hàng chứ không chỉ nạp job. Chỉ rõ đơn đã lưu nhưng PDF chưa có. 2. Mở consumer invoice trước, email sau; tải file và mở thư. Giữ cùng mã đơn để người nghe nối được các bước. 3. Khi nói chia việc, đặt nhịp 4s và thêm worker trong lúc backlog còn. Nếu để 0s có thể hoàn tất nhanh đến mức khó nhìn, dù nghiệp vụ vẫn thật. 4. Dùng email sai để nói về cô lập lỗi: một khách cần sửa thông tin không giữ các khách tốt trong work queue. 5. Dùng crash sau commit để nói về độ tin cậy: không chỉ “bật lại worker”, mà phải giải thích vì sao nhận lại không được tạo thêm hóa đơn. 6. Với Kafka, chỉ assignment/partition và chính sách xử lý lỗi. Thử luôn cách gỡ lỗi Kafka và đọc lại log; không dùng chênh lệch vài giây để kết luận sản phẩm nào nhanh hơn.

Ở máy chiếu nên zoom trình duyệt 110–125% nếu chữ trong log nhỏ. Đừng đọc toàn bộ JSON/event; chọn một mã đơn và ba đến năm mốc liên quan. Chỉ số Ready/Unacked có thể trễ lấy mẫu nên đối chiếu cùng log/worker, không đợi mọi ô đổi đồng thời.

Phạm vi chưa kiểm tra và rủi ro cần nói đúng

Chưa đo tải lớn, ngắt điện/đĩa, nhiều node, failover, lỗi mạng mọi thời điểm, email Internet hay gửi SMTP trùng trong mọi crash window. Bài này không kiểm thử việc người dùng học có hiểu hết hay dự đoán được điểm giảng viên cho. Bảo đảm trong demo chỉ đúng dữ liệu/cấu hình và các tình huống đã thử.

SMTP và DB không có transaction chung; email có thể lặp. Classic DLX một broker có giới hạn an toàn. Mọi dữ liệu dùng chung SQLite/file volume nên chưa phải phân ranh giới dịch vụ độc lập hoàn chỉnh. UI/API điều khiển process chỉ dùng cục bộ và một web process. Các lỗi tạm thời của SMTP/PDF hiện vào DLQ, cần thao tác retry; chưa có retry/backoff tự động theo nhóm lỗi.

Không cần thêm hệ thanh toán/tồn kho chỉ để demo “trông thật hơn”; chúng làm loãng cơ chế RabbitMQ. Với mục tiêu seminar, luồng đơn→PDF→email cộng backlog/lỗi/phục hồi đã có đầu vào, đầu ra và lý do kỹ thuật rõ ràng.

Phụ lục: mở đúng hàm, đúng dòng

Dòng được tính trực tiếp từ source khi build ngày 31/10/2026. Đổi code thì tạo lại PDF. Mã SHA-256 đầu mỗi file dùng đối chiếu phiên bản nội dung, không phải mật khẩu.

app.py | SHA-256 e34b59422960

lifespan - dòng 13-18

index - dòng 27-28

health - dòng 32-33

shop/api.py | SHA-256 6b03d1237cdc

dispatch - dòng 27-52

startup - dòng 55-58

shutdown - dòng 61-69

Item - dòng 72-75

Customer - dòng 78-81

Checkout - dòng 84-87

WorkerRequest - dòng 90-92

Seed - dòng 95-98

Delay - dòng 101-102

Fix - dòng 105-106

Replay - dòng 109-111

catalog - dòng 115-116

checkout - dòng 120-127

get_order - dòng 131-135

invoice_pdf - dòng 139-143

service_status - dòng 146-159

state - dòng 163-175

add_worker - dòng 182-192

crash_worker - dòng 196-204

observation - dòng 208-209

seed - dòng 213-224

fix_email - dòng 228-233

retry_task - dòng 237-263

kafka_dead_letter - dòng 267-272

duplicate - dòng 276-280

analytics_replay - dòng 284-320

read - dòng 285-316

reset - dòng 324-335

shop/store.py | SHA-256 ceeecd67356c

connect - dòng 27-36

init - dòng 39-74

log - dòng 77-83

email_valid - dòng 86-87

quote - dòng 90-108

enqueue - dòng 111-122

create_order - dòng 125-150

order - dòng 153-171

task - dòng 174-177

received - dòng 180-190

finish - dòng 193-195

invoice - dòng 198-201

commit_invoice - dòng 204-211

sent - dòng 214-216

record_sent - dòng 219-222

pending_outbox - dòng 225-229

published - dòng 232-237

publish_error - dòng 240-242

update_worker - dòng 245-253

stop_worker - dòng 256-258

settings - dòng 261-266

reset - dòng 269-273

fix_email - dòng 276-285

resolve - dòng 288-290

retry - dòng 293-300

snapshot - dòng 303-322

shop/broker.py | SHA-256 e75a93208745

connection - dòng 18-23

queue - dòng 26-27

dlq - dòng 30-31

topology - dòng 34-45

rabbit_publish - dòng 48-50

bootstrap - dòng 53-54

topic - dòng 57-59

ensure_kafka - dòng 62-80

kafka_publish - dòng 83-96

delivered - dòng 88-89

rabbit_metrics - dòng 99-112

shop/worker.py | SHA-256 b34b48c71519

complete - dòng 10-13

rabbit - dòng 16-51

consume - dòng 22-41

kafka - dòng 54-132

on_assign - dòng 67-71

on_revoke - dòng 72-75

work - dòng 79-83

shop/fulfillment.py | SHA-256 9646e6568b30

BusinessError - dòng 21-22

money - dòng 25-26

render_invoice - dòng 29-57

p - dòng 39-39

fulfill - dòng 60-109

static/shop.js: điểm vào từ giao diện

toast - dòng 47

api - dòng 54

busy - dòng 69

empty - dòng 79

badge - dòng 82

changeTab - dòng 85

renderCatalog - dòng 103

saveCart - dòng 145

renderCart - dòng 150

checkout-form.onsubmit - dòng 182

itemsTable - dòng 213

totals - dòng 216

renderOrders - dòng 219

showInvoice - dòng 266

liveWorkers - dòng 284

workerCards - dòng 289

dataTable - dòng 312

renderOperations - dòng 315

renderComparison - dòng 431

renderState - dòng 475

refresh - dòng 481

ensureWorkers - dòng 614

prepare - dòng 620

boot - dòng 696

Phụ lục: trích đoạn mã thật có số dòng

Các dòng dài được ngắt hiển thị để vừa A4; số bên trái là dòng gốc. Trích đoạn là phần để đối chiếu, không phải file Python độc lập để copy chạy.

HTTP 202 và lưu đơn | shop/api.py

```
119 | @router.post('/checkout',status_code=202)
120 | def checkout(req:Checkout):
121 |     start=time.perf_counter()
122 |     try:
123 |         oid,created=store.create_order(req.customer.model_dump(),[i.model_dump() for
     i in req.items],req.idempotency_key)
124 |     except ValueError as exc:
125 |         raise HTTPException(422,str(exc)) from exc
126 |     return dict(order_id=oid,created=created,status='accepted',api_ms=round((time.pe
    rf_counter()-start)*1000,1),
127 |                 meaning='Đơn hàng và outbox đã được lưu bền. Chỉ hiển thị queued sau
     khi broker xác nhận publish.',order=store.order(oid))
128 |
```

Idempotency và transaction order/outbox | shop/store.py

```
135 |     oid = 'DH-' + uuid.uuid4().hex
136 |     with connect() as db:
137 |         # BEGIN IMMEDIATE serializes conflicting idempotency keys across requests.
138 |         db.execute('BEGIN IMMEDIATE')
139 |         prior = db.execute('SELECT id,fingerprint FROM orders WHERE request_key=?',
    (request_key,)).fetchone()
140 |         if prior:
141 |             if prior['fingerprint'] != fingerprint:
142 |                 raise ValueError('Khóa gửi lại đã được dùng cho một giỏ hàng khác')
143 |             return prior['id'], False
144 |         db.execute('INSERT INTO orders VALUES(?,?,?,?,?,?,?,?,?,?,?)',
145 |                    (oid, request_key, fingerprint, transport, json.dumps(customer,
    ensure_ascii=False),
146 |                     json.dumps(priced['items'], ensure_ascii=False),
    priced['subtotal'], priced['shipping'], priced['total'],
147 |                     time.time(), experiment, int(crash_after_commit)))
148 |         task_id = enqueue(db, oid, 'invoice', transport)
149 |         log('ORDER_SAVED', oid, task_id, transport=transport, detail='Đơn hàng và
    outbox được lưu trong cùng transaction', db=db)
150 |     return oid, True
```

Invoice và outbox email cùng transaction | shop/store.py

```
204 | def commit_invoice(oid, number, filename, snapshot, worker):
205 |     with connect() as db:
206 |         row = db.execute('SELECT transport FROM orders WHERE id=?',
    (oid,)).fetchone()
207 |         cursor = db.execute('INSERT OR IGNORE INTO invoices VALUES(?,?,?,?,?)',
    (oid,number,filename,json.dumps(snapshot,ensure_ascii=False),time.time()))
208 |         if cursor.rowcount:
209 |             enqueue(db, oid, 'email', row['transport'])
210 |             log('INVOICE_COMMIT',oid,oid+':invoice',worker,row['transport'],'PDF +
    hóa đơn; tác vụ gửi email được thêm vào outbox trong cùng transaction',db=db)
211 |         return cursor.rowcount == 1
```

Topology, DLX và persistent publish | shop/broker.py

```
def topology(conn):
    ch = conn.channel()
    ch.exchange_declare(EXCHANGE, exchange_type='direct', durable=True)
    ch.exchange_declare(DLX, exchange_type='direct', durable=True)
    for role in ROLES:
        ch.queue_declare(dlq(role), durable=True, arguments={'x-queue-type':'classic'})
        ch.queue_bind(dlq(role), DLX, role + '.failed')
        ch.queue_declare(queue(role), durable=True, arguments={'x-queue-type':'classic',
            'x-dead-letter-exchange':DLX,'x-dead-letter-routing-key':role+'.failed',
            'x-max-length':10000,'x-overflow':'reject-publish'})
        ch.queue_bind(queue(role), EXCHANGE, role)
    return ch


def rabbit_publish(ch, payload):
    ch.basic_publish(EXCHANGE, payload['role'], json.dumps(payload).encode(),
        properties=pika.BasicProperties(delivery_mode=2,content_type='application/json',message_id=payload['task_id']),mandatory=True)
```

Nhận, xử lý, NACK hoặc ACK | shop/worker.py

```
            with bus.connection() as conn:
                ch=bus.topology(conn);ch.basic_qos(prefetch_count=1)
                def consume(channel,method,props,body):
                    payload=None
                    try:
                        payload=json.loads(body)
                        store.received(payload,name,'rabbitmq',method.redelivered)
                        fulfill(payload,name,'rabbitmq',conn.sleep)
                    except Exception as exc:
                        if payload and 'task_id' in payload:
                            store.finish(payload['task_id'],'dead_letter',str(exc))

store.log('NACK_DLQ',payload.get('order_id'),payload['task_id'],name,'rabbitmq',str(exc))
                        else:

store.log('INVALID_DLQ',worker=name,transport='rabbitmq',detail=type(exc).__name__)
                        channel.basic_nack(method.delivery_tag,requeue=False)
                    else:
                        # A lost connection while ACKing is not a business failure.
                        # Reconnect and let broker redelivery + idempotency recover.
                        channel.basic_ack(method.delivery_tag)
                        complete(payload,name,'rabbitmq')
                    finally:
                        store.update_worker(name,'rabbitmq',role,'idle')
                ch.basic_consume(bus.queue(role),consume,auto_ack=False)
                store.update_worker(name,'rabbitmq',role,'idle')
```

Dedup invoice và crash sau commit | shop/fulfillment.py

```
64 |         raise BusinessError('Không tìm thấy đơn hàng')
65 |     if role=='invoice':
66 |         prior=store.invoice(oid)
67 |         if prior:
68 |             if not (store.FILES/prior['filename']).is_file():
69 |                 render_invoice(order,prior['number'])
70 |             store.log('DEDUP',oid,payload['task_id'],worker,transport,'Hóa đơn đã
   có; không tạo lại side effect hoặc tác vụ email')
71 |             return
72 |         store.update_worker(worker,transport,role,'rendering',order_id=oid)
73 |         store.log('PDF_RENDER',oid,payload['task_id'],worker,transport,'Tính tiền
   theo snapshot giá server; tạo PDF chi tiết có font tiếng Việt')
74 |         pause(store.settings()['observation_delay'])
75 |         number='MS-
   '+datetime.fromtimestamp(order['created'],LOCAL_TZ).strftime('%Y%m%d')+'-'+oid[-
   8:].upper()
76 |         filename=render_invoice(order,number)
77 |         created=store.commit_invoice(oid,number,filename,order,worker)
78 |         if created and order['crash_after_commit']:
79 |
   store.log('CRASH_BEFORE_ACK',oid,payload['task_id'],worker,transport,'Hóa đơn và
   outbox email đã commit; process thoát trước xác nhận broker')
80 |             os._exit(42)
81 |     elif role=='email':
```

Attachment, SMTP và ghi sổ gửi | shop/fulfillment.py

```
98 |         message_id=f'<{oid}@moc-studio.test>'
99 |         message['Message-ID']=message_id
100 |         message.set_content(f"Xin chào {order['customer']['name']},\n\nĐơn hàng của
   bạn đã được xác nhận.\nTổng cộng: {money(order['total'])}.\nHóa đơn chi tiết nằm trong
    PDF đính kèm.\nThanh toán COD khi nhận hàng.\n\nĐây là email được gửi tới Mailpit cục
    bộ của seminar.")
101 |         message.add_attachment((store.FILES/invoice['filename']).read_bytes(),mainty
   pe='application',subtype='pdf',filename=invoice['number']+'.pdf')
102 |         with smtplib.SMTP(os.getenv('SHOP_SMTP_HOST','127.0.0.1'),int(os.getenv('SHO
   P_SMTP_PORT','1025')),timeout=8) as smtp:
103 |             refused=smtp.send_message(message)
104 |             if refused:
105 |                 raise BusinessError('SMTP từ chối người nhận')
106 |         # SMTP and SQLite are not one transaction. A crash here can resend email.
107 |         store.record_sent(oid,recipient,message_id,worker,transport)
108 |     else:
109 |         raise BusinessError('Loại tác vụ không được hỗ trợ')
```

Kafka xử lý rồi commit | shop/worker.py

```
78 |
79 |             def work(msg,payload,repeat=False):
80 |
   store.received(payload,name,'kafka',repeat,(msg.topic(),msg.partition(),msg.offset()))
81 |                 fulfill(payload,name,'kafka',time.sleep)
82 |                 consumer.commit(message=msg,asynchronous=False)
83 |                 complete(payload,name,'kafka')
84 |
```

Kafka pause partition sau lỗi | shop/worker.py

```
108 |                     raise RuntimeError(str(msg.error()))
109 |                 payload=json.loads(msg.value())
110 |                 try:
111 |                     work(msg,payload)
112 |                 except Exception as exc:
113 |                     # Explicit policy: preserve order on this partition. Other
114 |                     # partitions keep running. This is NOT a Kafka-wide freeze.
115 |                     store.finish(payload['task_id'],'blocked',str(exc))
116 |                     consumer.pause([TopicPartition(topic,msg.partition())])
117 |                     blocked[msg.partition()]=(msg,payload)
118 |
    store.log('PARTITION_BLOCKED',payload['order_id'],payload['task_id'],name,'kafka',
119 |                               f'partition={msg.partition()},
    offset={msg.offset()}: {exc}. Chưa commit; chờ sửa/retry hoặc DLQ do ứng dụng xây.')
120 |             if time.monotonic()-last_update>1:
121 |                 store.update_worker(name,'kafka',role,'blocked' if blocked else
    'idle' if assigned else 'unassigned',assigned,
122 |                                     next(iter(blocked.values()))[1]['order_id']
    if blocked else None)
```